Machine Learning for Customer Churn Prediction

Nelson David Posso Suarez
Javier Alejandro Penagos Hernandez
Nicolas Castro Rivera

1 Executive Summary

Customer Churn is a problem about customer discontinue a company's services or products. Machine learning techniques can be applied with purpose of predict if a customer has probability of be churn, this information can help the company to take decisions for not lost clients. This work search explore different machine learning techniques to compare and get the best approach to solve this problem.

2 Dataset Download and Integrity Verification

2.1 Dataset Source and Acquisition

The dataset is the **WSDM – KKBox Music Service Churn Prediction Challenge**, publicly available through the Kaggle competition platform at
`https://www.kaggle.com/competitions/kkbox-churn-prediction-challenge`.
The dataset was released by KKBox Co., Ltd. under the competition's terms of use, which permit academic and non-commercial research purposes. No paid subscription is required; a free Kaggle account is sufficient to access all files. To reproduce this experiment, the following two steps are required:

1. **Accept the competition terms:** Create a free Kaggle account, navigate to the competition page, and accept the data usage rules. This step is mandatory before any file can be downloaded.
2. **Download and place the raw files:** Download all nine `.csv.7z` archives provided by the competition and place them, **without extracting**, in the directory `workshops/data/`. The preprocessing pipeline extracts and converts them automatically during execution.

No further manual setup is required. The data pipeline reads the compressed archives directly from `workshops/data/` and writes the intermediate `.parquet` files to `workshops/data/parquet/`. The final processed cohorts are produced in `workshops/w1/export-tabular-v5/`.

2.2 Dataset Characteristics

The competition provides nine compressed files (`.csv.7z`). Table 1 summarises each file, its role in the pipeline, and its approximate uncompressed row count.

File	Description	Rows (approx.)	Columns
train.csv	February cohort labels: user ID and churn target	992,931	2
train_v2.csv	March cohort labels: user ID and churn target	970,960	2
members_v3.csv	User demographic profiles (age, gender, city, etc.)	6,769,473	7
transactions.csv	Earlier subscription transaction records	19,435,022	9
transactions_v2.csv	Recent subscription transaction records	21,591,521	9
user_logs.csv	Earlier daily listening behaviour logs	18,396,362	9
user_logs_v2.csv	Recent daily listening behaviour logs	18,396,362	9
sample_submission_v2.csv	Test set user IDs for competition submission	224,000	2
sample_submission_zero.csv	Baseline submission with all predictions set to zero	224,000	2

Table 1: KKBox dataset files and their characteristics.

2.3 Integrity Verification

The Kaggle platform does not publish official checksums for competition datasets. To guarantee that collaborators download the correct files, SHA-256 hashes were computed directly on the nine `.csv.7z` archives using the project script `workshops/w1/src/data_info/dataset_info.py`. The full hashes are stored in `workshops/data/checksums.json`; Table 2 shows the first 16 characters of each digest together with the compressed file size on disk.

File	SHA-256 (prefix)	Size (MB)
train.csv.7z	e2e7cec3392158ad	33.6
train_v2.csv.7z	683158156862e226	32.8
members_v3.csv.7z	a0e0b92b2ef9cc11	242.3
transactions.csv.7z	ff273f50f8e690c7	707.5
transactions_v2.csv.7z	78d01c8e90b86479	48.9
user_logs.csv.7z	2d21a83ddeeaf508	7,136.1
user_logs_v2.csv.7z	5e7b1d2b6fce3ff0	686.0
sample_submission_v2.csv.7z	329dfe694ccc76c7	30.7
sample_submission_zero.csv.7z	8d709c30a1e7dc52	32.8

Table 2: SHA-256 checksums of the raw `.csv.7z` archives (first 16 hex characters shown).

To verify the downloaded files, run the following command from the project root:

```
python workshops/w1/src/data_info/dataset_info.py
```

The script computes the SHA-256 of each archive in `workshops/data/`, compares the result against `checksums.json`, and prints `OK` or `FAIL` for each file. It exits with code 1 if any file is missing or does not match.

3 Exploratory Data Analysis

The KKBox dataset is composed of four distinct relational tables. Tables 3–6 describe the features present in each raw file, including a brief description and data type as stored in the source CSV.

Feature	Description	Data type
msno	Hashed subscriber identifier (primary key)	`str`
is_churn	Churn label: 1 if the user did not renew within 30 days of expiry, 0 otherwise	`int8`

Table 3: Schema of `train.csv` and `train_v2.csv`

Feature	Description	Data type
msno	Hashed subscriber identifier (primary key)	`str`
city	City of residence encoded as an integer code (1–22)	`int8`
bd	Subscriber age in years. Contains invalid values outside the range [1, 100]	`int16`
gender	Subscriber gender (male / female). Contains missing values	`str`
registered_via	Registration channel encoded as an integer code	`int8`
registration_init_time	Account registration date in YYYYMMDD format	`int32`
expiration_date	Membership expiration date in YYYYMMDD format. Not used as a model feature	`int32`

Table 4: Schema of `members_v3.csv`

Feature	Description	Data type
msno	Hashed subscriber identifier (foreign key)	`str`
date	Listening date in YYYYMMDD format	`int32`
num_25	Number of songs played less than 25% of their duration	`int32`
num_50	Number of songs played between 25% and 50% of their duration	`int32`
num_75	Number of songs played between 50% and 75% of their duration	`int32`
num_985	Number of songs played between 75% and 98.5% of their duration	`int32`
num_100	Number of songs played at least 98.5% of their duration (completed)	`int32`
num_unq	Number of unique songs played on that day	`int32`
total_secs	Total seconds of music listened to on that day	`float64`

Table 5: Schema of `user_logs.csv` and `user_logs_v2.csv`

Feature	Description	Data type
msno	Hashed subscriber identifier (foreign key)	`str`
payment_method_id	Payment method encoded as an integer code (2–41)	`int8`
payment_plan_days	Number of days in the purchased subscription plan	`int16`
plan_list_price	Listed price of the subscription plan in New Taiwan Dollars (NTD)	`int16`
actual_amount_paid	Amount actually paid by the subscriber in NTD	`int16`
is_auto_renew	Flag indicating automatic renewal (1 = yes, 0 = no)	`int8`
transaction_date	Date of the transaction in YYYYMMDD format	`int32`
membership_expire_date	Membership expiration date after the transaction in YYYYMMDD format	`int32`
is_cancel	Flag indicating subscription cancellation (1 = yes, 0 = no)	`int8`

Table 6: Schema of `transactions.csv` and `transactions_v2.csv`

3.1 Target Distribution

3.2 Feature Statistics

3.3 Data Quality

3.4 Feature Relationships

4 Preprocessing Pipeline

The preprocessing pipeline implemented in this work is partially based on the public analysis published by Asad Rizvi [1], which includes an exploratory data analysis and a feature engineering approach for the KKBox Churn Prediction Challenge. From his work we adopted the general structure for joining the relational tables and several feature engineering ideas at the transaction and listening level. However, his approach targeted a different modelling objective and proposed a larger feature set with a different temporal framing. We discarded a significant portion of his proposed features, simplified the listening aggregation scheme, and redesigned the cohort structure around a month-over-month temporal split that better reflects a real production scoring scenario.

The full pipeline is implemented in `workshops/w1/w1_data_preprocessing.ipynb`. The model training and evaluation that consumes the pipeline output is in `workshops/w1/w1_tabular_models.ipynb`.

4.1 Raw Extraction and Parquet Conversion

The nine `.csv.7z` archives in `workshops/data/` are extracted and converted to Parquet format by the `convert_raw_to_parquet()` function (`workshops/w1/src/data_processing/utils.py`), called in Section 2 of the preprocessing notebook. Each archive is decompressed one at a time using the `7z` binary, and only the columns defined in the project schema are retained and cast to compact integer or float types. The resulting files are written with `zstd` compression to `workshops/data/parquet/`. The conversion is idempotent: existing Parquet files are skipped, so the step can be safely resumed after an interruption.

Parquet was chosen over CSV because it is a columnar binary format that stores type information natively. Reading a single column from a large Parquet file does not require parsing the entire file, which reduces both I/O and memory usage significantly compared to CSV for the multi-million-row tables in this dataset.

4.2 Cohort Definition and User Index

The dataset is structured around two labelled cohorts defined in Section 3 of the preprocessing notebook:

- **February development cohort** (`train.csv`): 992,931 users with features cut at 2017-01-31 and churn labels for February 2017. Churn rate: 6.4%.
- **March test cohort** (`train_v2.csv`): 970,960 users with features cut at 2017-02-28 and churn labels for March 2017. Churn rate: 9.0%.

A shared integer index (`uid`) is assigned over the union of both cohorts (1,082,190 unique users), replacing the raw `msno` string for all subsequent joins. A user who appears in both months receives the same `uid` but a different cutoff, feature snapshot, and label, which mirrors monthly scoring in production.

4.3 Member Features

Static demographic features are extracted from `members_v3.parquet` in Section 4 of the preprocessing notebook. Invalid ages (outside the range [1, 100]) and negative registration channel values are imputed with the mean age and mode channel computed exclusively on the February cohort. Three derived features are added: `tenure_days` (days elapsed from the registration date to the feature cutoff), `registration_year`, and `registration_month`. A binary flag `bd_missing` marks rows where the original age was invalid or absent. This domain contributes **7 features** per user.

4.4 Transaction Features

Transaction records from both `transactions.parquet` and `transactions_v2.parquet` are concatenated and deduplicated in Sections 5–6 of the preprocessing notebook. Exact duplicates—rows identical across all eight transaction fields for the same user—are removed, keeping only the first occurrence. Per-record derived columns are then computed: plan duration, plan coverage ratio, discount (list price minus amount

paid), and behavioural flags. These records are subsequently aggregated into **one row per user**, yielding approximately 29 features covering counts, sums, averages, last-record attributes, and timing features relative to the cutoff (e.g., `days_to_expire`, `days_since_last_transaction`). All transaction history up to the cohort cutoff is used; no fixed time window is applied.

4.5 Listening Log Features

Daily listening logs from both `user_logs.parquet` and `user_logs_v2.parquet` are processed in Sections 7–8 of the preprocessing notebook. Records are restricted to the three calendar months ending at the feature cutoff and filtered to target users. Negative play counts are set to zero and non-finite `total_secs` values are replaced by zero. The resulting user-day table is grouped to monthly sums and active-day counts. A daily p99.9 clipping limit is applied to the per-day mean of each listening counter—these limits are fitted exclusively on the February cohort and reused for March to prevent leakage.

The last three active months for each user are then pivoted into a wide row (`last_*`, `last_second_*`, `last_third_*`). Several redundant per-channel columns are dropped, and four month-over-month trend features are added: `trend_songs`, `trend_secs`, `trend_ratio100`, and `trend_ratio25`. Recency features (`days_since_last_log`, `has_listening_logs`) are computed relative to the cutoff. This domain contributes approximately **23 features** per user.

4.6 Final Assembly and Saved Outputs

The three feature domains are assembled in Sections 9–10 of the preprocessing notebook by left-joining member, transaction, and listening features onto the user list. Users absent from a given domain receive zero for all count and sum columns; presence flags (`has_member_record`, `has_transactions`, `has_listening_logs`) mark whether a record existed. Table 7 lists the resulting **59 features** per user together with their source table.

Feature	**Description**	**Source**
city	City of residence (integer code)	members_v3
registered_via	Registration channel (integer code)	members_v3
bd	Subscriber age in years (imputed if invalid)	members_v3
bd_missing	Flag: 1 if original age was invalid or absent	members_v3
tenure_days	Days from registration to feature cutoff	members_v3
registration_year	Year of account registration	members_v3
registration_month	Month of account registration	members_v3
has_member_record	Flag: 1 if a member record exists for the user	Pipeline

Continued on next page

Feature	Description	Source
is_cancel_change_flag	1 if any transaction was a cancellation	transactions
is_auto_renew_change_flag	1 if any transaction had auto-renew enabled	transactions
transaction_count	Total number of transactions	transactions
average_amount_paid	Mean amount paid across all transactions	transactions
average_amount_charged	Mean listed price across all transactions	transactions
average_plan	Mean plan duration in days	transactions
n_payment_methods	Number of distinct payment methods used	transactions
n_plans	Number of distinct plan durations purchased	transactions
n_list_prices	Number of distinct listed prices seen	transactions
n_amounts_paid	Number of distinct amounts actually paid	transactions
duration	Mean duration (expiry minus transaction date) in days	transactions
plan_coverage_ratio	Mean ratio of duration to plan days	transactions
sum_plan_days	Total plan days purchased	transactions
sum_list_price	Total listed price across all transactions	transactions
sum_amount_paid	Total amount paid across all transactions	transactions
more_than_30	Count of transactions with duration $>$ 30 days	transactions
autorenew_not_cancel	Min flag: auto-renew on and not cancelled	transactions
not_autorenew_cancel	Min flag: auto-renew off and cancelled	transactions
max_plan_days	Maximum plan duration across all transactions	transactions
last_payment_method_id	Payment method of the most recent transaction	transactions
last_plan_days	Plan days of the most recent transaction	transactions
last_is_auto_renew	Auto-renew flag of the most recent transaction	transactions
last_is_cancel	Cancel flag of the most recent transaction	transactions
last_discount	Discount (list price minus paid) of the most recent transaction	transactions
days_to_expire	Days from last expiry date to cutoff (negative = already expired)	transactions

Continued on next page

Feature	Description	Source
expire_dayofweek	Day of the week of the last expiry date	transactions
days_since_last_transaction	Days from last transaction date to cutoff	transactions
difference	Mean amount paid minus mean amount charged	transactions
has_transactions	Flag: 1 if any transaction record exists	Pipeline
sum_last_month_unq	Sum of unique songs in the last active month	user_logs
sum_last_month_total_secs	Sum of listening seconds in the last active month	user_logs
mean_last_month_total_secs	Mean daily listening seconds in the last active month	user_logs
last_login	Number of active days in the last active month	user_logs
last_sum_total_songs	Total songs played in the last active month	user_logs
last_ratio_100	Ratio of fully-played songs in the last active month	user_logs
last_ratio_25	Ratio of barely-played songs in the last active month	user_logs
sum_last_second_month_total_secs	Sum of listening seconds in the second-to-last active month	user_logs
last_second_login	Number of active days in the second-to-last active month	user_logs
last_second_sum_total_songs	Total songs played in the second-to-last active month	user_logs
last_second_ratio_100	Ratio of fully-played songs in the second-to-last active month	user_logs
last_second_ratio_25	Ratio of barely-played songs in the second-to-last active month	user_logs
last_third_sum_total_songs	Total songs played in the third-to-last active month	user_logs
last_third_ratio_100	Ratio of fully-played songs in the third-to-last active month	user_logs
last_third_ratio_25	Ratio of barely-played songs in the third-to-last active month	user_logs

Continued on next page

Feature	Description	Source
login_difference	Difference in active days between last and second-to-last month	user_logs
trend_songs	Ratio of total songs last month to second-to-last month	user_logs
trend_secs	Ratio of listening seconds last month to second-to-last month	user_logs
trend_ratio100	Change in fully-played ratio from second-to-last to last month	user_logs
trend_ratio25	Change in barely-played ratio from second-to-last to last month	user_logs
days_since_last_log	Days from the last listening date to the cutoff	user_logs
has_listening_logs	Flag: 1 if any listening record exists	Pipeline

Table 7: Final feature set (59 features) with source table.

The two cohort tables are exported to `workshops/w1/export-tabular-v5/` as `train_feb.parquet` (February, 992,931 users) and `test.parquet` (March, 970,960 users). Imputation and clipping parameters are persisted in `preprocess_params.json` and the complete feature catalogue in `metadata.json`. Split indices for the development and test partitions are saved to `workshops/w1/artifacts/sherlock_w1/split_indices`

References

[1] Rizvi, A. (2018). *WSDM – KKBox's Churn Prediction Challenge*. Medium. Retrieved from `https://asad-99rizvi.medium.com/wsdm-kkboxs-churn-prediction-challenge-d7c9cfa21cbd`